

Sequence Diagram

Prova finale di Ingegneria del Software

Matteo Aldrigo - Gabriele Bertirotti - Filippo Baldissara

June 20, 2025

Contents

1	Protocollo di comunicazione	2
2	Fasi di gioco	2
2.1	Game lobby	2
2.2	Ship construction	3
2.3	Fix ship	6
2.4	Populate ship	7
2.5	Card round	7
2.6	Insufficient players	8

1 Protocollo di comunicazione

Il protocollo di comunicazione è basato sull'interfaccia *VirtualView*, implementata sia per RMI che per Socket. Quando viene invocato uno dei metodi di questa interfaccia, la gestione della comunicazione di rete è affidata al protocollo stesso. In particolare:

- **RMI:** L'invocazione avviene direttamente sul server, aggiungendo un parametro *UUID* che consente di identificare in modo univoco il client che ha effettuato la richiesta.
- **Socket:** Per ogni richiesta, viene costruito e serializzato un pacchetto specifico, tramite le implementazioni di *Message*, che viene inviato al server.

Il *GameModel* mantiene lo stato attuale della partita, ma sono i singoli stati che specificano quali comandi possono essere eseguiti in un dato momento. Ogni stato, infatti, implementa solo i metodi che risultano leciti in quella fase della partita; gli altri comandi, se invocati, lanciano eccezioni, impedendo l'esecuzione di azioni non permesse o fuori sequenza. Questo approccio previene l'invio di comandi errati da parte di client malevoli o malfunzionanti. Inoltre, tutti i messaggi ricevuti dalla rete vengono sanitizzati per verificarne la correttezza.

Nel caso in cui il server riceva parametri errati o incoerenti rispetto allo stato corrente del gioco, l'evento viene notificato esclusivamente al client che ha generato l'azione, permettendogli di correggere l'errore e ripetere l'invio.

Si noti che, quando a seguito di un'azione vengono restituiti sia la risposta al comando sia il prossimo stato, nei diagrammi questi sono rappresentati come due messaggi distinti esclusivamente per ragioni di chiarezza grafica. Nella reale implementazione, invece, viene utilizzato l'oggetto *Answer*, che permette di includere sia lo *state* (la risposta al comando), sia l'eventuale *nextState*, ossia il prossimo stato in cui il programma si è evoluto.

I seguenti diagrammi mostrano il flusso di funzionamento di una possibile partita. Sebbene vengano rappresentati il leader e altri giocatori in generale, il diagramma si riferisce al caso specifico di una partita tra due giocatori, sia per semplicità sia per chiarezza nell'illustrazione delle invocazioni tra client e server.

2 Fasi di gioco

2.1 Game lobby

In questa fase iniziale del gioco, ogni giocatore ha la possibilità di:

- **Creare una nuova partita:** il leader sceglie il proprio nickname, seleziona il colore e configura la partita impostando il livello di difficoltà e il numero di giocatori che potranno partecipare.
- **Unirsi a una partita in fase di avvio:** il giocatore visualizza tutte le partite in attesa di giocatori e può scegliere a quale unirsi, impostando il proprio nome e colore.
- **Aggiornare la lista delle partite disponibili:** tramite un'apposita azione è possibile aggiornare l'elenco delle partite in attesa di giocatori.
- **Riconnettersi a una partita precedente:** in caso di disconnessione, il giocatore può reinserire il nickname scelto in precedenza per essere riportato allo stato attuale della partita da cui era stato disconnesso.

Nota bene: Per brevità, la fase iniziale di connessione dei client al server non viene descritta, in quanto standard è ormai consolidata.

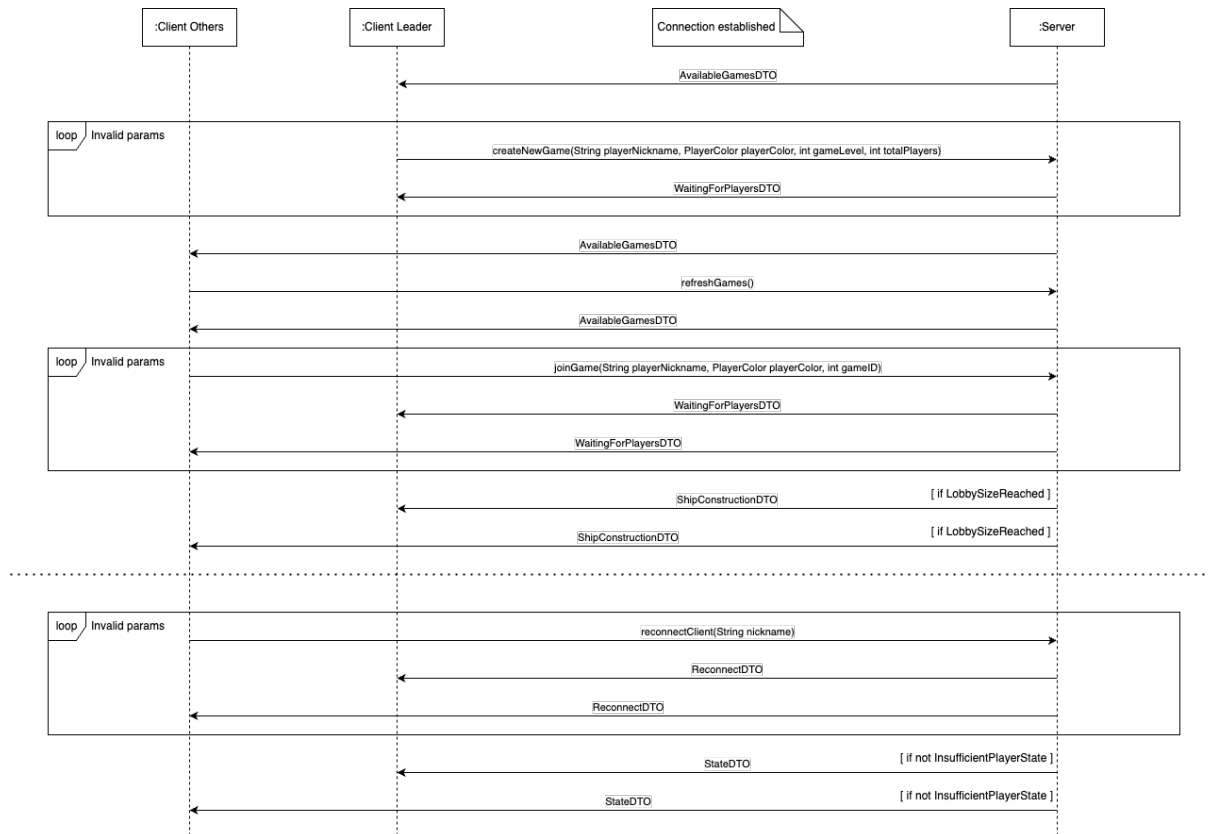


Figure 2.1: Game Lobby

2.2 Ship construction

Durante questa fase, ciascun giocatore ha la possibilità di costruire la propria nave e di effettuare altre azioni. In particolare, questa fase è divisa in due passi:

1. **Selezione del Componente:** Il giocatore può scegliere un componente con cui potrà andare a costruire la propria nave.
2. **Costruzione della Nave:** Il componente selezionato nel passo precedente è ora in mano al giocatore che lo ha selezionato, il quale può ruotarlo, riservarlo, deselezionarlo oppure piazzarlo sulla propria nave.

I comandi disponibili, divisi per sezione, sono i seguenti:

1. Selezione del Componente:

- **Select Tile:** Permette di selezionare un componente dal mucchio condiviso tra tutti i giocatori.
- **Select Reserved Tile:** Permette di selezionare un componente che in precedenza era stato messo nel mucchio dei componenti riservati per essere utilizzato in futuro.
- **Finish Ship:** Comunica al *server* che il player corrente ha deciso di finire la costruzione della propria nave in anticipo e, una volta inviato tale messaggio, quest'ultimo si mette in attesa che tutti gli altri giocatori che stanno ancora costruendo finiscano (oppure che scada prima il tempo rimanente con la clessidra).
- **Fast Build:** (Comando a scopo dimostrativo) Permette a un giocatore di utilizzare come nave una di quelle preconfigurate presenti sul server. A quel punto quest'ultimo la invia al giocatore ed egli si mette in attesa.
- **Flip Timer:** Permette ai giocatori di capovolgere la clessidra quando viene comunicato che è possibile effettuare tale azione.

- **Visualize Subdeck:** Permette a un giocatore di visualizzare uno dei tre mazzi di carte messi a disposizione dal gioco per la visualizzazione. Inoltre, un mazzo può essere visualizzato da un solo giocatore alla volta.
- **Visualize Ships:** Permette ai giocatori di visualizzare lo stato delle navi degli altri.

2. Costruzione della Nave:

- **Deselect Tile:** Permette a un giocatore di deselezionare il componente che ha attualmente selezionato. Tale azione è permessa se quest'ultimo è stato pescato dal mucchio di componenti messo in comune a tutti i giocatori, altrimenti tale componente proviene dal mucchio di quelli riservati e, stando alle regole del gioco, un componente riservato non può essere più rimesso nel mucchio in comune.
- **Reserve Tile:** Permette a un giocatore di mettere nel mucchio dei componenti riservati il componente che sta attualmente contemplando. Tale azione è possibile solo se lo stesso giocatore ha meno di 2 componenti riservati, altrimenti è costretto a deselezionare il componente e rimetterlo nel mucchio comune.
- **Place Selected Tile:** Permette a un giocatore di piazzare il componente che sta attualmente contemplando. Se decide di fare ciò, gli verranno chieste le coordinate di dove piazzarlo e, se valide, il componente viene saldato permanentemente al telaio della nave.
- **Rotate Right:** Permette a un giocatore di ruotare a destra (quindi in senso orario) il componente che ha attualmente selezionato.
- **Rotate Left:** Permette a un giocatore di ruotare a sinistra (quindi in senso antiorario) il componente che ha attualmente selezionato.

I *sequence diagram* delle due sezioni che compongono la fase di *ship construction* sono disponibili qui:

1. Selezione del Componente $\rightarrow$ 2.2)

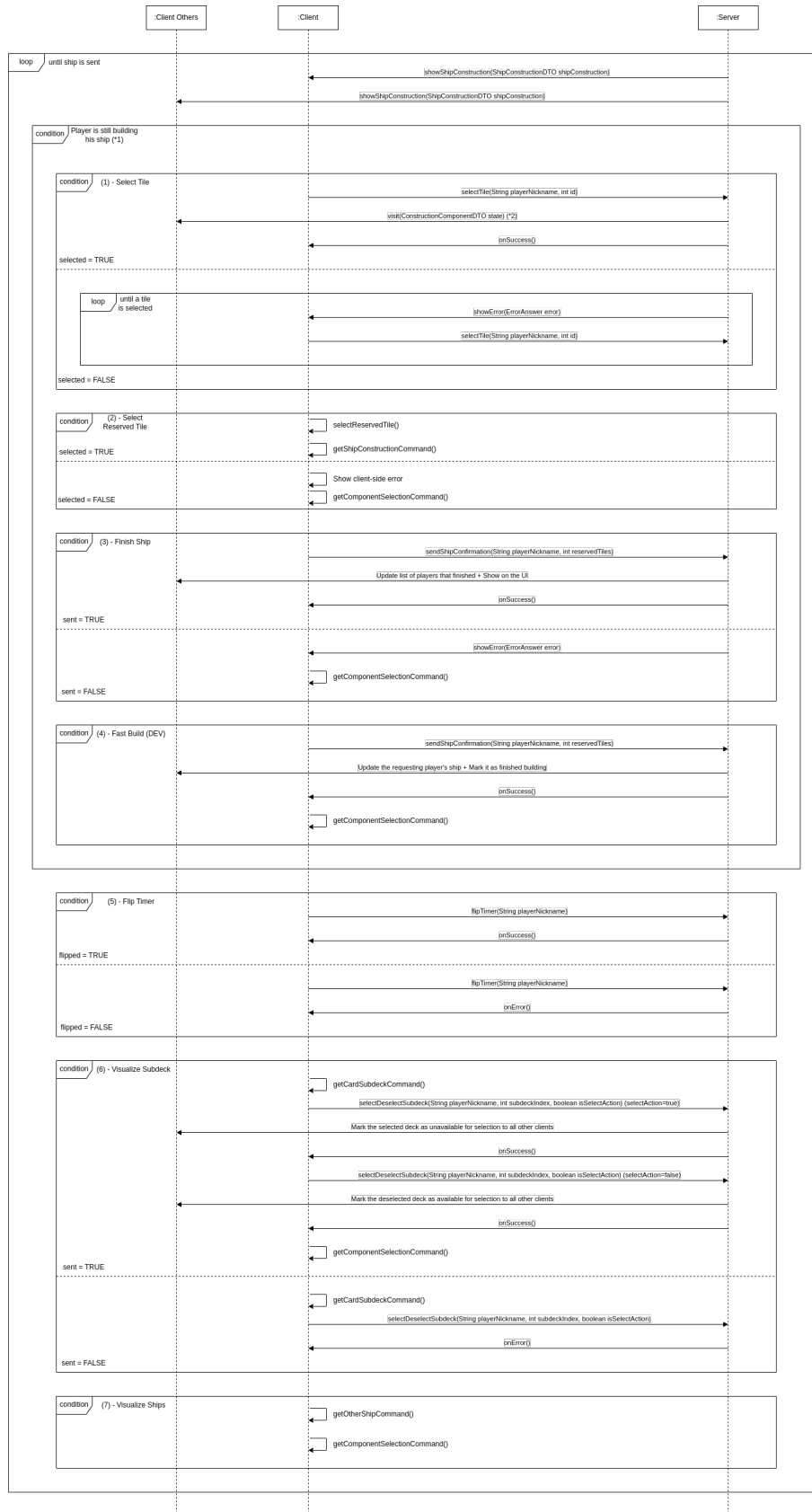


Figure 2.2: *Flow* dei comandi nella fase di Selezione del Componente

2. Costruzione della Nave → 2.3)

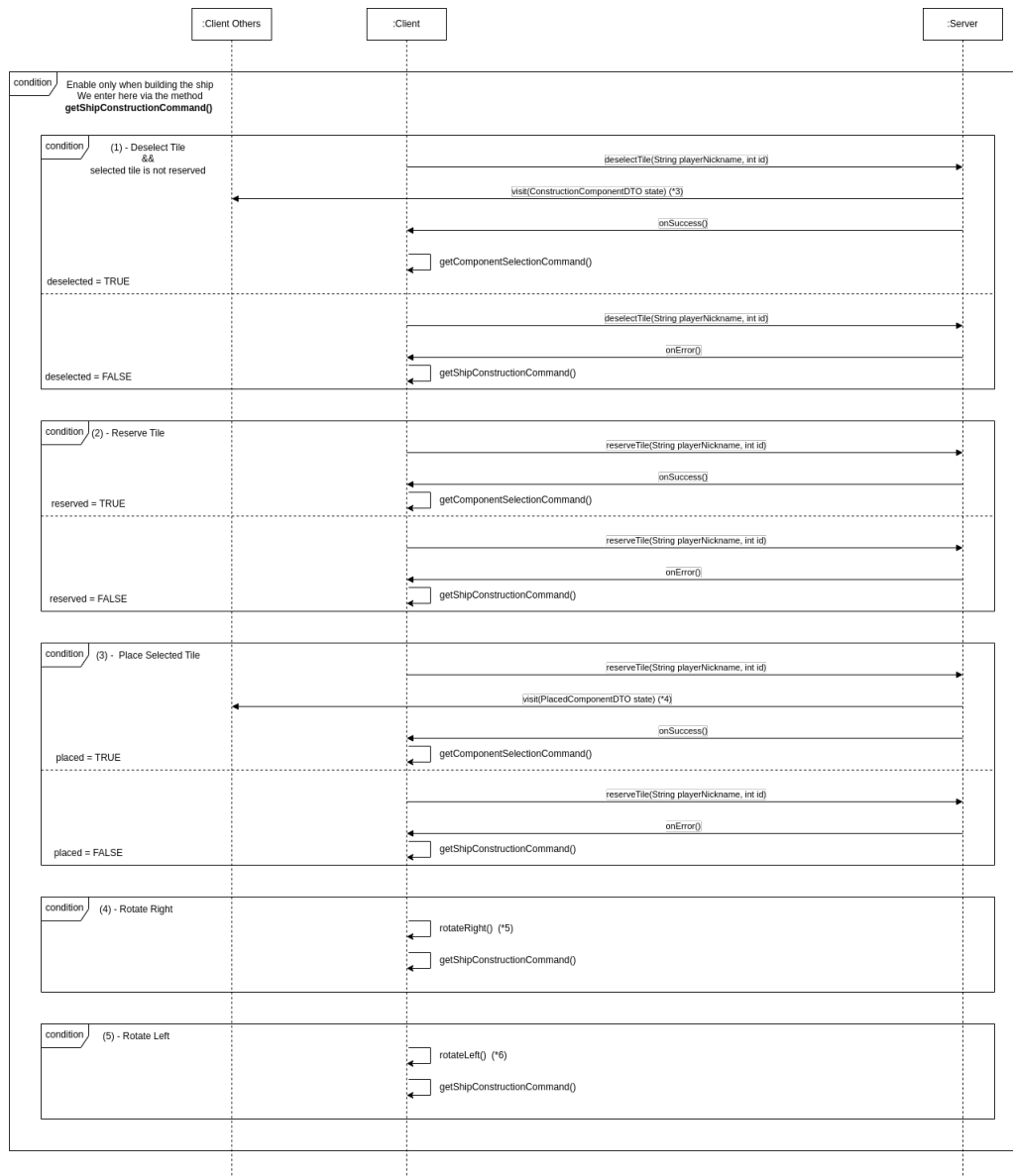


Figure 2.3: *Flow* dei comandi nella fase di Costruzione della Nave

2.3 Fix ship

Questa fase segue direttamente alla fase di ship construction solo nel caso ci sia almeno un giocatore con una nave non valida. In questa fase ogni giocatore con una nave che presenta delle irregolarità dovrà iniziare uno scambio di messaggi col server al fine di rendere valida la propria nave. Una volta ricevuto un componente da rimuovere il server manderà la modifica a tutti i giocatori presenti in partita, segnalando al giocatore che ha mandato il messaggio se la sua nave ha bisogno di ulteriori modifiche; nel caso la nave sia valida il giocatore resta in attesa fino a che anche gli altri giocatori non hanno sistemato le irregolarità nelle loro navi. I giocatori che all'inizio della fase possiedono una nave valida, riceveranno solamente gli aggiornamenti riguardo alle altre navi, senza dover inviare al server alcun messaggio.

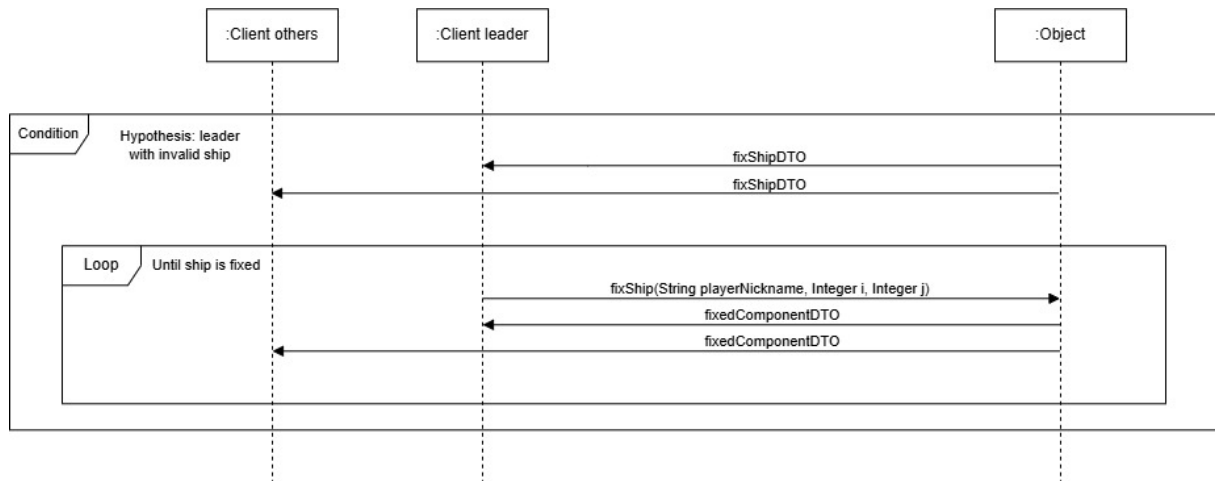


Figure 2.4: Fix ship

2.4 Populate ship

In questa fase, nella quale si entra solo nel caso vi sia almeno una nave che necessita di occupare dei posti vacanti, ogni giocatore che presenta una nave con cabine vuote inizia uno scambio di messaggi col server, fino a che la sua nave non è totalmente occupata. I messaggi di ritorno dal server, che arrivano a tutti i giocatori, specificano che tipo di forma di vita è stata inserita in una cabina specificata, segnalando se la nave del client che ha mandato il messaggio è piena; in tal caso il giocatore che ha inviato il messaggio attenderà il riempimento delle navi restanti da parte degli altri giocatori.

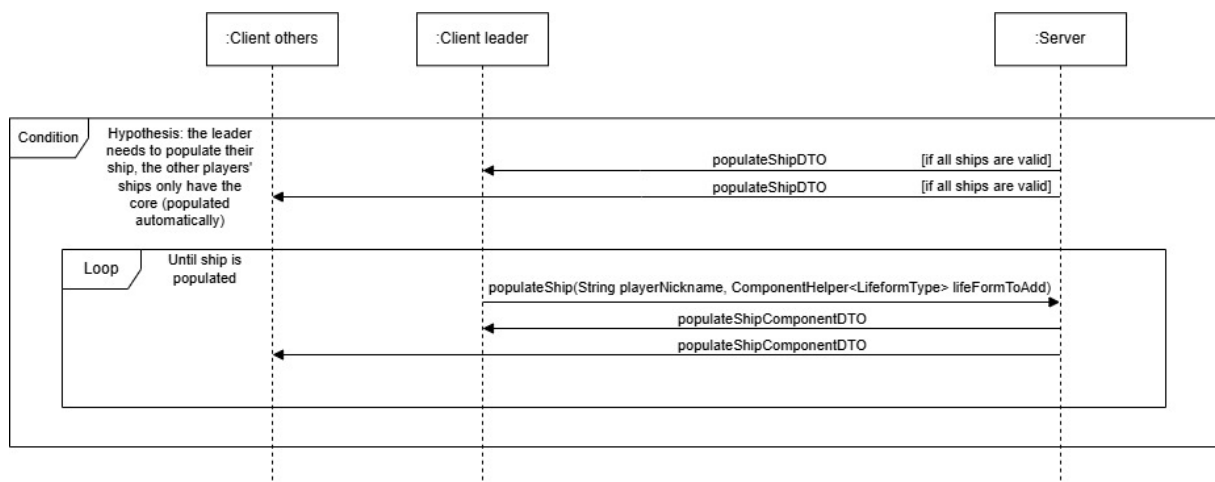


Figure 2.5: Populate ship

2.5 Card round

Il *Card Round* rappresenta la fase del gioco in cui i vari giocatori interagiscono con le carte inviate loro dal server. Quando arriva il proprio turno ogni giocatore invia al server le azioni compiute riguardo la carta giocata; in risposta, il server ritorna a tutti i server le informazioni riguardo i vari cambiamenti da applicare alle navi (o ai giocatori stessi), e altre informazioni come la carta corrente. Lo scambio di messaggi continua fino all'esaurimento delle carte giocabili, oppure nel caso vengano eliminati abbastanza giocatori.

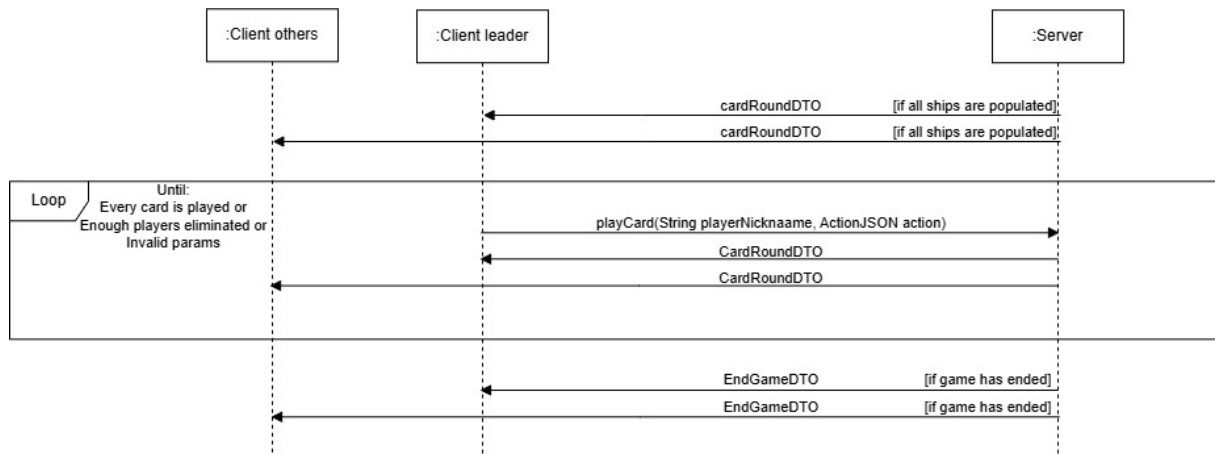


Figure 2.6: Card round

2.6 Insufficient players

Per soddisfare la funzionalità aggiuntiva di *resilienza alle disconnessioni*, è stato implementato un meccanismo automatico di *ping* che consente al server di verificare se un client risulta disconnesso. Se, nell'arco di 15 secondi, il server non riceve alcun ping dal giocatore interessato, quest'ultimo viene segnato come disconnesso. Se il numero di giocatori connessi diventa minore o uguale a 1, il sistema entra nello stato di *InsufficientPlayers*. In questa situazione possono verificarsi due scenari:

- Il giocatore si riconnette e la partita riprende dal punto in cui era stata interrotta.

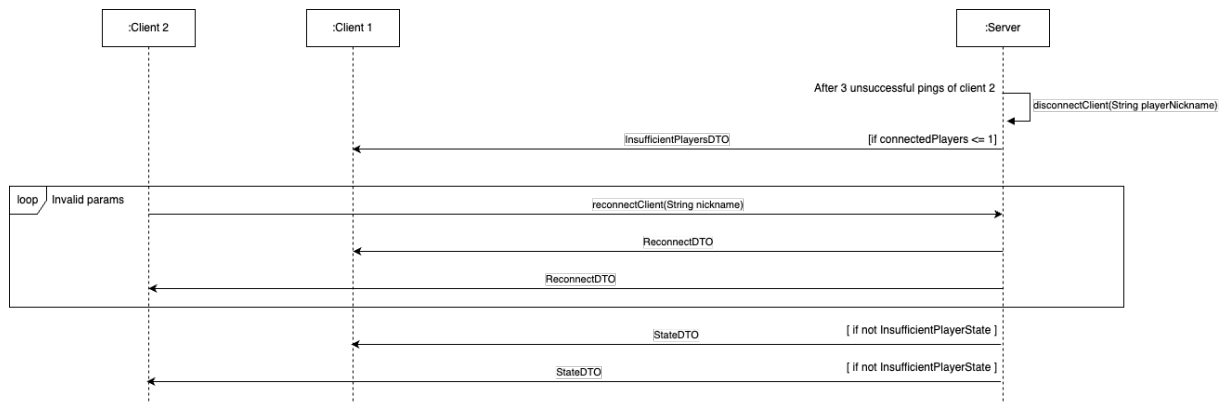


Figure 2.7: Insufficient player with successful reconnection

- Il giocatore non si riconnette entro il tempo massimo prestabilito e la partita termina, facendo evolvere il gioco nello stato finale di *EndGame*.

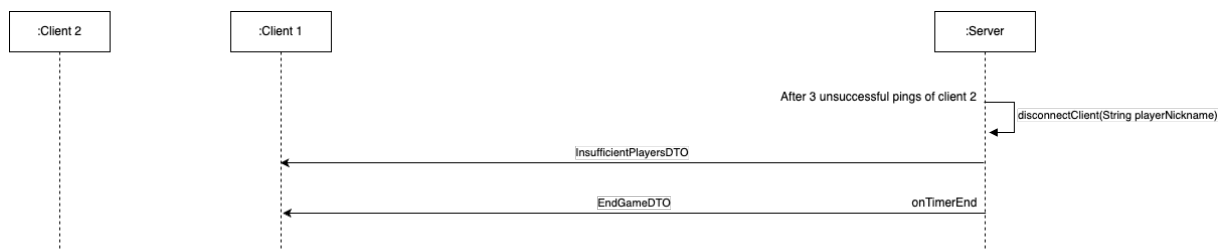


Figure 2.8: Insufficient player with expired timer